Types, Initialization and Casts

Initialization forms and brace traps, `auto` and `decltype`, implicit conversions, `enum class`, the four named casts, `const` / `constexpr` / `static` and what C++14 to 20 added.

Initialization and brace traps

```
int a;          // default-initialized: INDETERMINATE value, never read it
int b{};        // value-initialized: zero
int c = 5;      // copy-initialized
int d(5);       // direct-initialized
int e{5};       // list-initialized: the preferred form, rejects narrowing
// int f{3.7};  // ERROR: narrowing double → int inside braces
int g = 3.7;    // compiles: silently truncates to 3
```

KIND	EXAMPLE	NOTES
aggregate	`Point p{1, 2};`	no constructors: members set in order
designated (C++20)	`Point p{.x = 3, .y = 4};`	names the members, order must match
value-initialize	`Point p{};`	every member zero

BRACES PREFER `INITIALIZER_LIST`

```
std::vector<int> v1(3, 7);   // parentheses: 3 elements, each 7
std::vector<int> v2{3, 7};   // braces: the initializer_list
constructor wins → the two elements 3 and 7
```

If a class has an `initializer_list` constructor, `{ ... }` picks it even when another constructor matches better (compiled: `Widget w{1, 2}` called the list constructor, `Widget w(1, 2)` the (int, int) one). Empty braces `{}` call the default constructor.

- **Initialize everything.** An uninitialized local is **indeterminate**; reading it is undefined behaviour.
- Braces catch accidental narrowing at compile time: a free safety check.
- A `static_cast` in braces (`int{x}`) is a conversion that must not narrow.

`auto` , `decltype` and modern syntax

```
int x = 0;
const int& r = x;
auto a = r;                 // int           auto drops const and the reference
decltype(r) b = x;          // const int&    decltype gives the exact declared
type
decltype(auto) c = r;       // const int&    decltype(auto) = "deduce like
auto, but keep it exact"

auto t = {1, 2, 3};         // std::initializer_list<int>, NOT a vector
```

AUTO TRAPS

`auto y = v[0];` *copies*: write `const auto&` to avoid it. Same for `for (auto [k, v] : map)` (a copy of each pair). `auto` hides what a function returns: fine for iterators and lambdas, poor for public interfaces.

FEATURE	INSTEAD OF	EXAMPLE
Range-based `for`	index / iterator loops	`for (const auto& x : v) {}`
`nullptr`	`NULL` / `0`	`int* p = nullptr;`
`using` alias	`typedef`	`using Map = std::map<int, std::string>;`
`override` / `final`	hoping you matched the signature	`void f() override;`
`= default` / `= delete`	empty bodies, private unimplemented copy	`Foo(const Foo&) = delete;`
`static_assert`	runtime checks of compile-time facts	`static_assert(sizeof(int) == 4);`
Structured bindings (C++17)	`.first` / `.second`	`for (auto& [key, val] : m) {}`
`if` with initializer (C++17)	variable leaking into outer scope	`if (auto it = m.find(k); it ≠ m.end()) {}`
`[[nodiscard]]`	ignored error codes	`[[nodiscard]] bool open();`

Implicit conversions and `enum class`

The compiler converts silently between arithmetic types. Compiled results of the classic surprises:

EXPRESSION	RESULT	WHY
`-1 < 1u`	**false**	the signed value is converted to unsigned and becomes huge
`uint8_t(200) + uint8_t(100)`	300, stored back as 44	small types are promoted to `int` before arithmetic; storing back wraps
`7 / 2`	3	integer division; `7 / 2.0` is 3.5
`static_cast<int>(-3.9)`	-3	conversion to an integer truncates toward zero
`char` 0xFF widened to `int`	-1 (signed `char`)	whether plain `char` is signed is implementation-defined

- **Usual arithmetic conversions:** smaller than `int` becomes `int`; mixing signed and unsigned of the same rank converts to unsigned; `int` with `double` becomes `double`.
- **Never compare signed with unsigned** (`-Wsign-compare`); use `std::ssize` or cast deliberately.
- Single-argument constructors are implicit conversions: mark them `explicit` (OOP lesson).

```
enum Color { Red, Green };                // plain enum: leaks names into the scope, converts to int silently
int n = Green;                            // compiles

enum class Level { Info = 1, Error = 2 };// scoped: Level::Error, no implicit conversion
int m = static_cast<int>(Level::Error);   // an explicit cast is required (compiled: 2)
```

Prefer `enum class`, with an explicit underlying type when the size matters: `enum class Side : std::uint8_t { Buy, Sell };`

Casts and type conversion

Four named casts replace the C cast `(T)x`: each states intent and is searchable.

CAST	USE FOR	DANGER
`static_cast<T>(x)`	well-defined conversions: `int` to `double`, base to derived pointer when you are sure	low
`dynamic_cast<T>(x)`	checked downcast in a polymorphic hierarchy. Returns `nullptr` (pointer) or throws `std::bad_cast` (reference) on failure	runtime cost; base needs a virtual function
`const_cast<T>(x)`	add or remove `const` (legacy APIs)	modifying a truly const object is UB
`reinterpret_cast<T>(x)`	reinterpret raw bits (pointer to integer, unrelated pointer types)	very high, rarely correct

```
double avg = static_cast<double>(total) / count;       // avoids integer division

Animal* a = getAnimal();
if (auto* d = dynamic_cast<Dog*>(a)) {                  // safe downcast: null if it is not a Dog
    d->fetch();
}
```

> **RULE** Many casts mean a design problem. Prefer virtual functions to `dynamic_cast` chains; no C casts.

`const`, `constexpr` and `static`

- `const`: unchangeable after initialization. Use everywhere.
- `constexpr`: computable **at compile time** (array sizes, template arguments), and still callable at runtime.

```
constexpr int square(int x) { return x * x; }
constexpr int N = square(4);          // 16, computed by the compiler
std::array<int, square(3)> a;         // size 9: legal because it is a
constant expression
int runtime = readInput();
int r = square(runtime);              // also fine: same function runs at
runtime

static_assert(N == 16, "math is broken");   // checked at compile time
```

`static` means three things:

WHERE	MEANING
Local variable in a function	one instance that lives for the whole program and keeps its value between calls
Global / namespace scope	internal linkage: visible only inside this `.cpp` file
Class member	belongs to the class, shared by all objects

Linkage and `extern "C"`

C++ **mangles** function names (it encodes the parameter types so that overloads get different symbols); C does not. To call C code, or to be called by it, tell the compiler to use C linkage:

```
extern "C" int compareInts(const void* a, const void* b);      // an
unmangled symbol, callable from C

std::qsort(arr, 5, sizeof(int), compareInts);                   // a C API
taking a callback (compiled: sorted 12345)

extern "C" {                                                    // a whole
header of C declarations
    #include "legacy.h"
}
```

- An `extern "C"` function cannot be overloaded and cannot be a template or a member.
- Pointers to `extern "C"` functions are technically a different type from C++ function pointers; compilers accept them for C callbacks in practice.
- **Linkage rules:** a non-`static`, non-`inline` function defined in a header and included twice gives a "multiple definition" link error; mark it `inline`. `extern` on a variable declares it without defining it.

What C++14, 17 and 20 added

A map of which standard added what, so you know which flag a feature needs:

STANDARD	FEATURE	ONE-LINER
C++11	move semantics, smart pointers, lambdas, `auto`, `constexpr`, `std::thread`, variadic templates, `nullptr`, range-for	The big modernisation.
C++14	generic lambdas, `make_unique`, return type deduction, binary literals `0b101`	Polishes C++11.
C++17	`std::optional`, `std::variant`, `std::string_view`, structured bindings, `if constexpr`, fold expressions, `std::filesystem`, parallel algorithms, `std::scoped_lock`, CTAD	Vocabulary types and compile-time branching.
C++20	concepts, ranges, `std::span`, `operator⟺`, coroutines, modules, `std::jthread`, `std::format`	Constraints on templates, composable algorithms.

The C++17 and C++20 vocabulary types (`optional`, `variant`, `span`, `string_view`) have their own lesson: Vocabulary Types. Full lists: modern-cpp-features, modern-cpp-tutorial. Your notes: Module 3.

FURTHER READING

C++ Primer (Lippman, Lajoie, Moo)

- 2.2 Variables (PDF pages 70 to 78)
- 2.5 Dealing with types (PDF pages 96 to 100)

- 2.1 Primitive built-in types (PDF pages 61 to 69)
- 4.11 Type conversions (PDF pages 188 to 194)
- 19.3 Enumerations (PDF pages 861 to 863)
- 6.5 Features for specialized uses (PDF pages 265 to 270)

Programming: Principles and Practice (Stroustrup)

- 2.8 to 2.10 Type safety, conversions, type deduction with auto (PDF pages 56 to 60)
- 8.4 to 8.6 Evolving a class (Date), enumerations, operator overloading (PDF pages 238 to 249)

Effective Modern C++ (Meyers)

- Item 2, Understand auto type deduction (PDF pages 36 to 41)
- Item 5, Prefer auto to explicit type declarations (PDF pages 55 to 61)
- Item 15, Use constexpr whenever possible (PDF pages 115 to 121)